JavaScript Assignments

Session 08 to 13 | Naveen AutomationLabs | 2026 Playwright Batch G1

Session 08 - 2D Arrays, Array Reverse, Objects, Object References

Covers: Multi-dimensional arrays, reversing arrays, creating objects, key-value pairs, dot and bracket notation, nested objects, object references.

Q1. What is a 2D array? Write an example of a 2D array with 3 rows and 2 columns.

Q2. How do you access the element at row 1, column 0 in a 2D array called matrix?

Q3. Write a program to print all elements of a 2D array using nested for loops.

Q4. What does the reverse() method do? Does it change the original array?

Q5. Write code to reverse the array [10, 20, 30, 40, 50] and print the result.

Q6. What is a JavaScript object? How is it different from an array?

Q7. Create an object called 'user' with properties: name, age, city, and isActive.

Q8. How do you access a property using dot notation and bracket notation? Show both.

Q9. How do you add a new property to an existing object? How do you delete one?

Q10. What is a nested object? Write an object 'user' that has an 'address' object inside it with flat, zip, and city.

Q11. What does user.address.flat return if address is a nested object inside user?

Q12. What happens when you assign one object variable to another? Do they share the same reference or create a copy?

Session 09 - Object Iteration (for...in), JSON.stringify, JSON.parse, Null Reference

Covers: for...in loop, Object.keys(), serialization (JS object to JSON), deserialization (JSON to JS object), null object reference, optional chaining.

Q13. How do you get all the keys of a JavaScript object? Which method do you use?

Q14. Write a for...in loop to print all keys and values of an object.

Q15. What is the difference between for...in and for...of?

Q16. What does JSON.stringify() do? What type does it return?

Q17. What does JSON.parse() do? What type does it return?

Q18. Write code to convert a JS object {name:'Tom', age:30} to a JSON string and print it.

Q19. Write code to convert the JSON string back to a JS object and access the name property.

Q20. What is serialization? What is deserialization?

Q21. What happens when you set an object to null and then try to access its property?

Q22. What is the ?. (optional chaining) operator? How does it help with null objects?

Q23. Given a nested object user with address inside it, how would you iterate over the address keys inside a for...in loop?

Q24. After JSON.stringify and JSON.parse, is the new object the same reference as the original? Explain.

Session 10 - Functions, Heap, Stack, Memory Allocation

Covers: Function basics, function types, parameters vs arguments, return values, call stack, heap memory, call by value.

Q25. What is a function in JavaScript? Why do we use functions?

Q26. Write a simple function called greet that prints 'Hello World' and call it.

Q27. What is the difference between parameters and arguments?

Q28. Write a function that takes two numbers as input, adds them, and returns the result.

Q29. What does a function return if there is no return statement?

Q30. What does typeof return when used on a function name?

Q31. What is call by value? Give a simple example.

Q32. What is the call stack? What happens when a function is called?

Q33. What is heap memory? What kind of data is stored there?

Q34. Write a function getStudentMarks that takes a student name and returns their marks using if-else. Return -1 if the student is not found.

Q35. What does functionName.length return?

Q36. What does functionName.name return?

Session 11 - Duplicate Functions, Default Parameters, Function Expression, Object with Functions, this Keyword

Covers: Function overloading (not supported), default parameter values, function expressions, objects with methods, this keyword.

Q37. What happens if you write two functions with the same name in JavaScript?

Q38. Does JavaScript support function overloading? Explain.

Q39. What are default parameters in a function? Write an example.

Q40. What happens if you pass null as a value for a default parameter?

Q41. What is a function expression? How is it different from a function declaration?

Q42. Write a function expression and assign it to a variable called cart.

Q43. Can you call a function expression before it is defined? Why or why not?

Q44. How do you add functions (methods) inside a JavaScript object?

Q45. Write an object called loginPage with properties username and password, and a method login() that prints the username.

Q46. What does the 'this' keyword refer to inside an object method?

Q47. Write an object with two methods where one method calls the other using 'this'.

Q48. What does functionExpression.name return? Does it return the variable name or the actual function name?

Session 12 - Null Handling, Rest Parameter, IIFE, Anonymous Function, Arrow Function

Covers: Optional chaining, rest parameters (...), IIFE pattern, anonymous functions, arrow function syntax.

Q49. What is the rest parameter in JavaScript? What symbol is used for it?

Q50. Write a function that accepts any number of arguments using the rest parameter and prints them all.

Q51. Can you use the rest parameter along with regular parameters? If yes, where should the rest parameter be placed?

Q52. What is an anonymous function? Give one example.

Q53. Where are anonymous functions commonly used?

Q54. What is an arrow function? Write a simple arrow function that adds two numbers.

Q55. What is the difference between a regular function and an arrow function in terms of syntax?

Q56. Write an arrow function with no parameters that prints 'Hello JS'.

Q57. Write an arrow function with one parameter that returns the value multiplied by 10.

Q58. What is an IIFE? What does it stand for?

Q59. Write an IIFE that prints 'Server started'.

Q60. Can you pass arguments to an IIFE? Write an example that takes a browser name and prints it.

Session 13 - Callback Functions

Covers: Callback functions, passing functions as arguments, callback with utility functions, forEach, filter with callbacks.

Q61. What is a callback function?

Q62. Write a simple example where one function is passed as an argument to another function.

Q63. Write a calculator function that takes two numbers and a callback, and returns the result of the callback.

Q64. Using the calculator function above, call it with add, subtract, multiply, and divide callbacks.

Q65. What is the difference between call by value, call by reference, and call by function (callback)?

Q66. Write a function perform that takes two callback functions and calls both of them.

Q67. How does forEach use a callback function? Write an example with an array of numbers.

Q68. How does filter use a callback function? Write an example to filter numbers greater than 3 from [1,2,3,4,5].

Q69. Can you pass an anonymous arrow function directly as a callback? Write an example.

Q70. Write a function launchBrowser that takes a browser name and a callback. The callback should use a switch-case to print which browser to launch and return true or false.

Q71. Write a callback-based calculator that uses rest parameters to accept any number of values and adds them all.

Q72. What happens if the callback function is not passed to a function that expects one?

Answer Key

Session 08 - 2D Arrays, Array Reverse, Objects, Object References

Q1. A 2D array is an array of arrays. Example: let matrix = [[1, 2], [3, 4], [5, 6]];

Q2. matrix[1][0] -- this gives the value 3.

Q3. let matrix = [[1,2],[3,4],[5,6]]; for(let i=0; i<matrix.length; i++) { for(let j=0; j<matrix[i].length; j++) { console.log(matrix[i][j]); } }

Q4. reverse() reverses the order of elements in an array. Yes, it modifies the original array in place.

Q5. let arr = [10, 20, 30, 40, 50]; arr.reverse(); console.log(arr); // [50, 40, 30, 20, 10]

Q6. An object stores data as key-value pairs. An array stores data as an ordered list with numeric indexes. Objects use named keys, arrays use numeric indexes.

Q7. let user = { name: 'Tom', age: 30, city: 'LA', isActive: true };

Q8. Dot notation: user.name -- Bracket notation: user['name']. Both return the same value.

Q9. Add: user.email = 'tom@gmail.com'; -- Delete: delete user.age;

Q10. let user = { name: 'Tom', age: 30, address: { flat: 101, zip: 545454, city: 'LA' } };

Q11. It returns the value of the flat property inside the nested address object, for example 101.

Q12. They share the same reference. Changing one will affect the other. Example: let u1 = {name:'Tom'}; let u2 = u1; u2.name = 'Jerry'; console.log(u1.name); // Jerry

Session 09 - Object Iteration (for...in), JSON.stringify, JSON.parse, Null Reference

Q13. Use Object.keys(objectName). It returns an array of all keys. Example: Object.keys(user) gives ['name', 'age', 'city'].

Q14. let user = {name:'Tom', age:30}; for(let key in user) { console.log(key, ':', user[key]); }

Q15. for...in is used to iterate over object keys (properties). for...of is used to iterate over iterable values like arrays and strings.

Q16. JSON.stringify() converts a JavaScript object into a JSON string. It returns a value of type 'string'.

Q17. JSON.parse() converts a JSON string back into a JavaScript object. It returns a value of type 'object'.

Q18. let user = {name:'Tom', age:30}; let jsonStr = JSON.stringify(user); console.log(jsonStr); // '{"name":"Tom","age":30}'

Q19. let jsonStr = '{"name":"Tom","age":30}'; let obj = JSON.parse(jsonStr); console.log(obj.name); // Tom

Q20. Serialization is converting a JS object to a JSON string (JSON.stringify). Deserialization is converting a JSON string back to a JS object (JSON.parse).

Q21. You get a TypeError. For example: let obj = {name:'Tom'}; obj = null; console.log(obj.name); // TypeError: Cannot read properties of null.

Q22. Optional chaining (?.) returns undefined instead of throwing an error if the object is null or undefined. Example: obj?.name returns undefined instead of a TypeError.

Q23. for(let key in user) { if(key === 'address') { for(let innerKey in user[key]) { console.log(innerKey, ':', user[key][innerKey]); } } }

Q24. No. After stringify and parse, the new object is a completely separate copy. Changing the new object will not affect the original.

Session 10 - Functions, Heap, Stack, Memory Allocation

Q25. A function is a reusable block of code that performs a specific task. We use functions to avoid code duplication and keep code organized.

Q26. function greet() { console.log('Hello World'); } greet();

Q27. Parameters are the variable names listed in the function definition. Arguments are the actual values passed when calling the function.

Q28. function add(a, b) { return a + b; } let result = add(10, 20); console.log(result); // 30

Q29. It returns undefined.

Q30. typeof functionName returns 'function'.

Q31. Call by value means a copy of the value is passed to the function. Changing the value inside the function does not affect the original. Example: function change(x) { x = 100; } let a = 5; change(a); console.log(a); // still 5.

Q32. The call stack is a data structure that keeps track of function calls. When a function is called, it is pushed onto the stack. When it finishes, it is popped off.

Q33. Heap memory is used to store objects and reference types. Objects, arrays, and functions are stored in heap memory.

Q34. function getStudentMarks(name) { if(name==='ravi') return 90; else if(name==='uday') return 80; else { console.log(name + ' not found'); return -1; } }

Q35. It returns the number of parameters the function expects (defined in the function signature).

Q36. It returns the name of the function as a string.

Session 11 - Duplicate Functions, Default Parameters, Function Expression, Object with Functions, this Keyword

Q37. The second function overwrites the first one. Only the latest function definition is used when you call it.

Q38. No. JavaScript does not support function overloading. If you define multiple functions with the same name, only the last one is kept.

Q39. Default parameters have a preset value if no argument is passed. Example: function login(role = 'admin') { console.log(role); } login(); // prints 'admin'.

Q40. The parameter receives null, not the default value. null is a valid value, so the default is not used.

Q41. A function expression assigns a function to a variable. Unlike function declarations, function expressions are not hoisted. Example: let greet = function() { console.log('hi'); };

Q42. let cart = function addToCart(product) { console.log('add to cart', product); return true; };

Q43. No. Function expressions are not hoisted. You will get a ReferenceError if you try to call it before the declaration line.

Q44. Define functions directly as properties of the object. Example: let obj = { greet() { console.log('hello'); } };

Q45. let loginPage = { username: 'naveen@gmail.com', password: 'naveen@123', login() { console.log('login using: ' + this.username); } };

Q46. 'this' refers to the current object that the method belongs to. It is used to access other properties and methods of the same object.

Q47. let page = { login() { console.log('login'); this.logout(); }, logout() { console.log('logout'); } }; page.login();

Q48. It returns the actual function name, not the variable name. Example: let cart = function addToCart(){}; console.log(cart.name); // 'addToCart'.

Session 12 - Null Handling, Rest Parameter, IIFE, Anonymous Function, Arrow Function

Q49. The rest parameter allows a function to accept any number of arguments as an array. It uses the ... (three dots) symbol.

Q50. function printAll(...values) { for(let e of values) { console.log(e); } } printAll('a', 'b', 'c');

Q51. Yes. The rest parameter must always be the last parameter. Example: function fillValues(name, ...details) { console.log(name, details); }

Q52. An anonymous function is a function without a name. Example: let greet = function() { console.log('hello'); };

Q53. Anonymous functions are commonly used as callback functions, in event handlers, and assigned to variables as function expressions.

Q54. An arrow function is a shorter way to write functions using the => syntax. Example: let add = (a, b) => a + b;

Q55. Arrow functions do not use the 'function' keyword. They use => instead. For single expressions, you can skip the curly braces and return keyword.

Q56. let print = () => console.log('Hello JS'); print();

Q57. let multiply = (n) => n * 10; console.log(multiply(5)); // 50

Q58. IIFE stands for Immediately Invoked Function Expression. It is a function that runs as soon as it is defined.

Q59. (function() { console.log('Server started'); })();

Q60. Yes. (function(browserName) { console.log('Browser:', browserName); })('chrome');

Session 13 - Callback Functions

Q61. A callback function is a function that is passed as an argument to another function and is called (invoked) later inside that function.

Q62. function sayHello(callback) { callback(); } function greet() { console.log('hi'); } sayHello(greet);

Q63. function calculator(a, b, callback) { return callback(a, b); }

Q64. let add=(a,b)=>a+b; let sub=(a,b)=>a-b; let mul=(a,b)=>a*b; let div=(a,b)=>a/b; console.log(calculator(10,20,add)); console.log(calculator(100,50,sub)); console.log(calculator(10,30,mul)); console.log(calculator(10,2,div));

Q65. Call by value passes a copy of the value. Call by reference passes the object reference (objects, arrays). Call by function (callback) passes a function itself as an argument to another function.

Q66. function perform(cb1, cb2) { cb1(); cb2(); } perform(() => console.log('hi'), () => console.log('hello'));

Q67. forEach calls the callback once for each element. Example: let nums = [1,2,3,4,5]; nums.forEach(e => console.log(e + 5));

Q68. filter returns a new array with elements that pass the callback condition. Example: let nums = [1,2,3,4,5]; let result = nums.filter(e => e > 3); console.log(result); // [4, 5]

Q69. Yes. Example: calculator(10, 20, (a, b) => a + b);

Q70. function launchBrowser(name, callback) { return callback(name); } let flag = launchBrowser('chrome', (b) => { switch(b.toLowerCase()) { case 'chrome': console.log('launch chrome'); return true; case 'firefox': console.log('launch firefox'); return true; default: return false; } });

Q71. function calculator(cb, ...nums) { console.log(cb(...nums)); } let addition = (...a) => { let sum=0; for(let i=0;i<a.length;i++){sum+=a[i];} return sum; }; calculator(addition, 1, 2, 3, 4); // 10

Q72. It will throw a TypeError: callback is not a function. Because the parameter will be undefined and you cannot call undefined as a function.